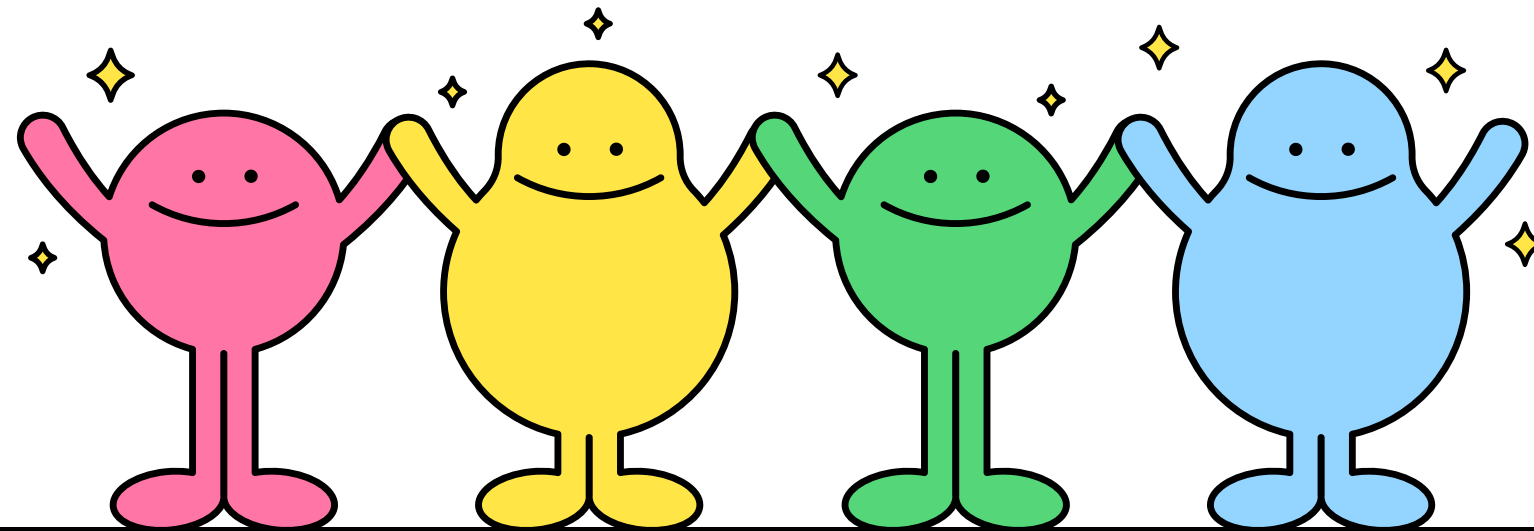


빌드 자동화로 개발 생산성 극대화하기

효율적인 C/C++ 개발의 시작, Makefile

2023664016 김채린



발표 목차

01 컴파일의 고통

02 Makefile의 등장

03 핵심 원리 : Incremental Build

04 기본 구조 : 3요소

05 실전 예시

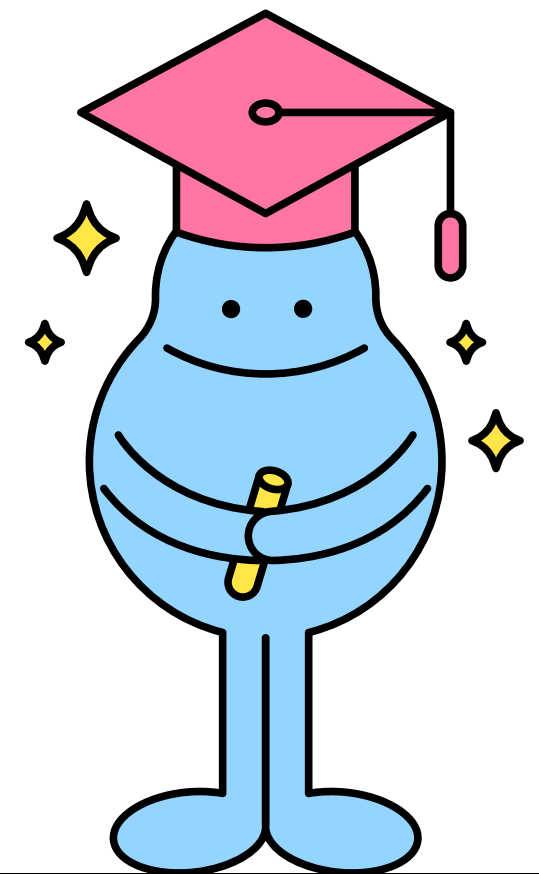
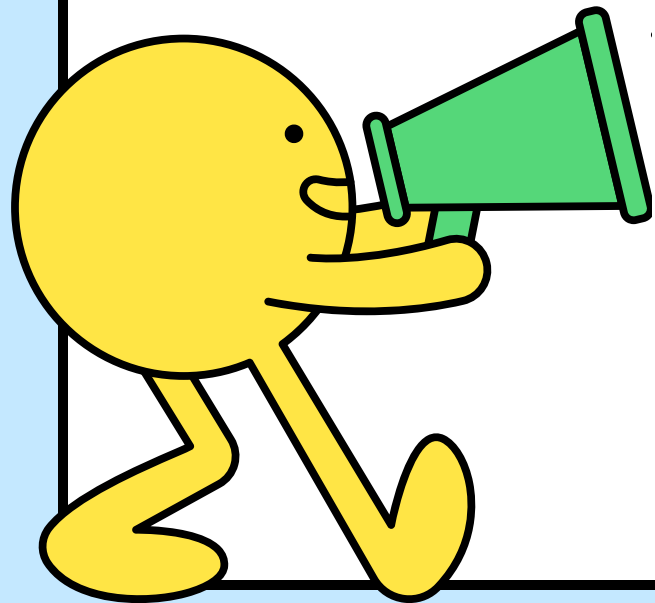
06 요약 및 Q&A

소스 파일 10개, 100개를 일일이 수동으로 빌드하시겠습니까?

반복적인 명령어 입력으로 인한
휴먼 에러(Typo) 발생

어떤 파일을 수정했는지 잊어버려
불필요한 전체 재컴파일 반복

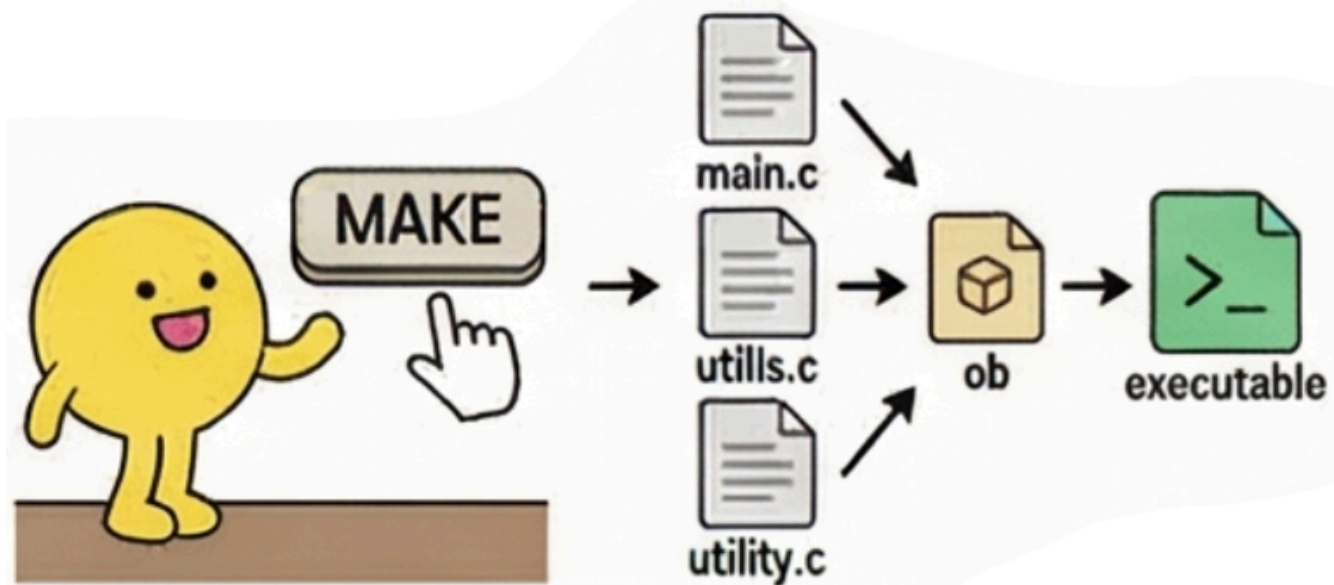
빌드 설정 공유가 안 되어 팀원 간 환경 불일치 발생



02 Makefile의 등장

복잡한 빌드 시나리오를 파일 하나에 담다

- 정의 : 소스 파일 간의 관계와 빌드 명령어를 기술한 설정 파일.
- 기능 : MAKE 명령어 하나로 컴파일부터 링크까지 자동 수행.
- 장점 : 누구나 같은 규칙으로 빌드 가능 (협업 효율 극대화).



단 한 줄의 명령어로 끝내는 빌드 자동화

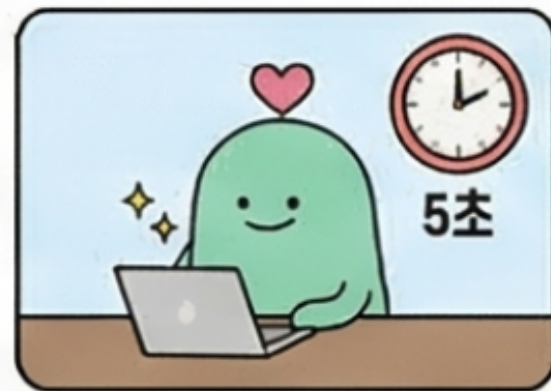
03 핵심 원리 : Incremental Build

바뀐 놈만 골라낸다! Incremental Build

왜 Makefile은 일반 셸 스크립트보다 빠른가?



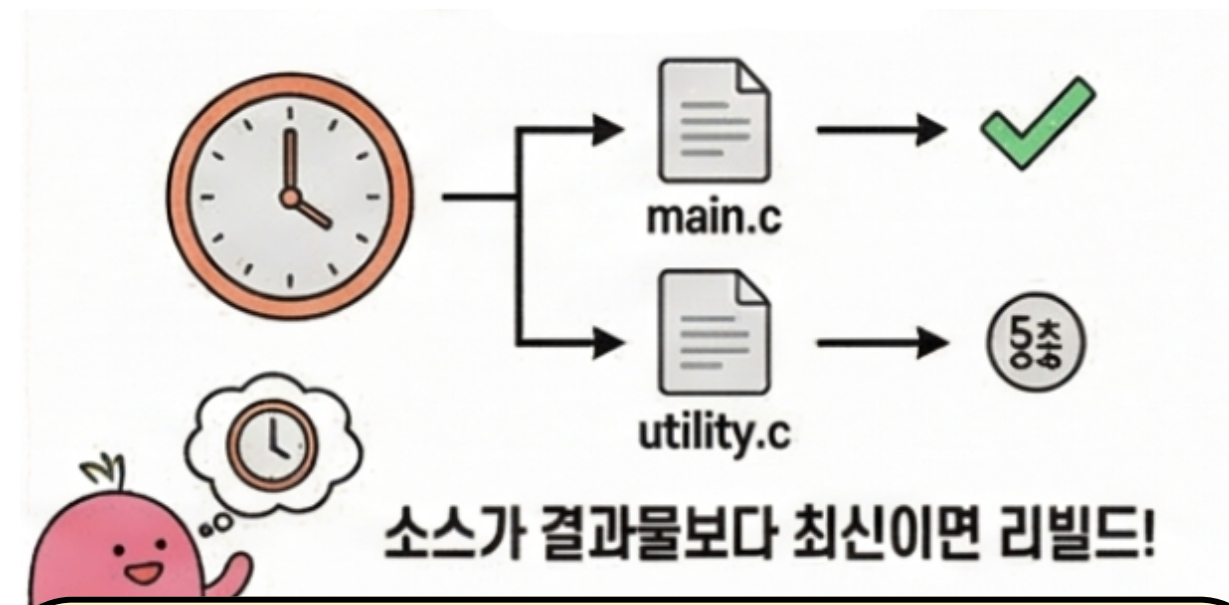
전체 다시 빌드
(셸 스크립트)



변경된 파일만 빌드
(make)

타임스탬프(Timestamp) 비교

소스 파일(*.C)과 목적 파일(*.O)의
수정 시간 확인



판단 로직

소스가 목적 파일보다 최신(NEWER)일 때만
해당 부분 재컴파일

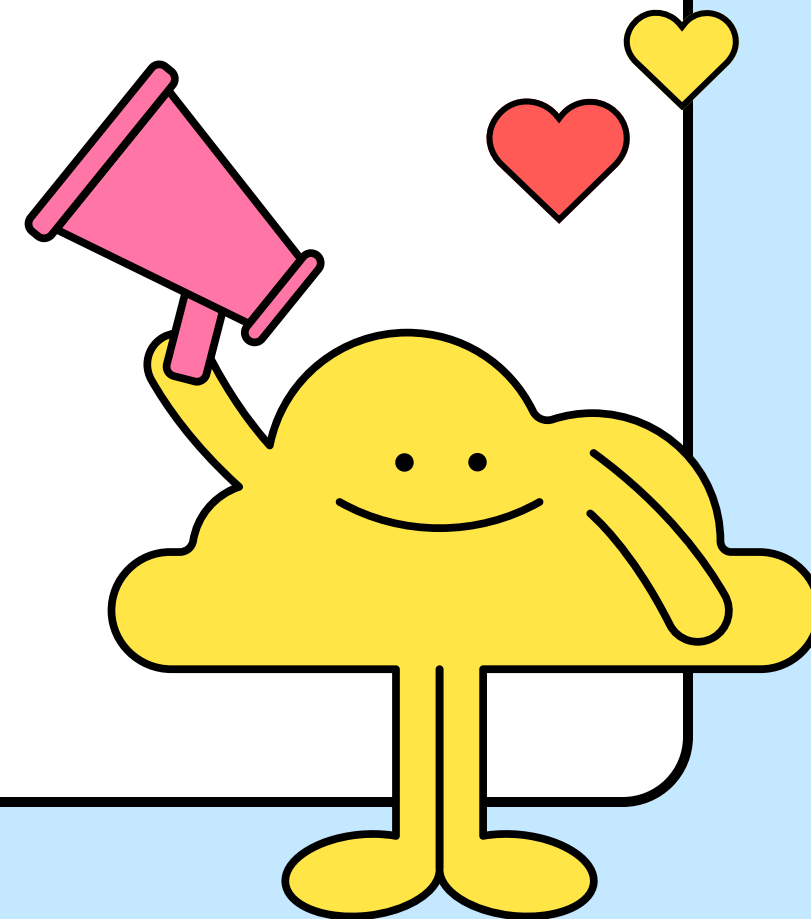
결과 : 대규모 프로젝트에서 빌드 시간 획기적 단축 !

Target, Dependencies, 그리고 Tab

- Target (타겟)
생성할 결과물 (예: 실행 파일명)
- Dependencies (의존성)
타겟을 만들기 위해 필요한 재료 파일들
- Recipe (레시피)
실제 실행할 명령어

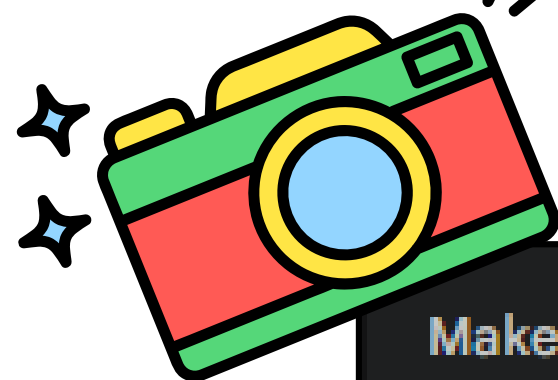


반드시 앞부분은
TAB으로 들여쓰기!



간결하지만 강력한 자동화 코드

가장 보편적인 Makefile의 형태



Makefile

```
CC = gcc           # 컴파일러 변수
CFLAGS = -Wall -g  # 컴파일 옵션 변수

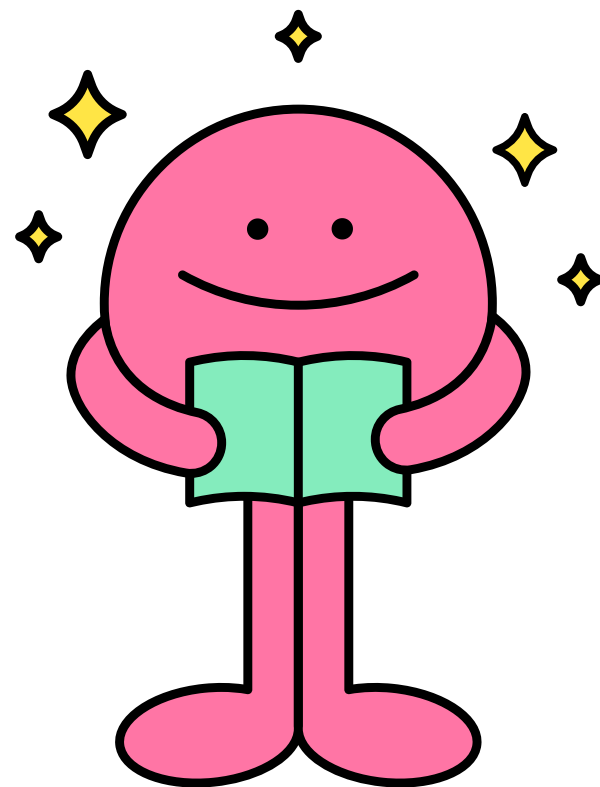
app: main.o utils.o # 타겟 : 의존성
    $(CC) $(CFLAGS) -o app main.o utils.o # 레시피

clean:              # 가짜 타겟 (Phony Target)
    rm -f *.o app
```

포인트 : make clean을 통해 언제든지 빌드 환경을 초기화할 수 있음



개발자의 시간을 아끼는 마법의 도구



- 효율성: 수정된 부분만 빌드하여 시간 절약.
- 일관성: 동일한 빌드 환경 제공.
- 전문성: 대규모 프로젝트 관리를 위한 필수 역량



감사합니다!

<https://www.canva.com/>

<https://gemini.google.com/>

<https://notebooklm.google/>

